

MovEasy — Architecture, Workflow & Operations Guide

Document version: 1.0 (February 2026)
Repository: MOVEASY-WEBSITE (React SPA + Firebase)

1. Executive summary

MovEasy is a **single-page application (SPA)** served as static files. The **browser** runs the UI; **Firebase** provides authentication, a document database (Firestore), file storage, optional Cloud Functions, and hosting. There is **no traditional Node/Express API server** in the main user path—data access is **direct from the client to Firestore** under **security rules**, which is the critical control plane for who can read or write what.

2. Technology map

Layer	Technology	Role
Language (UI)	JavaScript (ES modules) + JSX	All React components and business logic in the browser
UI framework	React 19	Components, state, effects
Routing	React Router v7 (BrowserRouter)	Public pages, role-gated /admin, /seller, /customer, staff /crm
Build / dev	Vite 8	Dev server, HMR, production bundle to dist/
Styling	Tailwind CSS 3 + some inline styles (e.g. CRM)	Layout, marketing pages
Maps	Leaflet + React-Leaflet	Interactive listing map
Animation	GSAP, Framer Motion	Marketing motion
Auth & data	Firebase JS SDK v12	auth, firestore, storage, functions
Error reporting (client)	Sentry (@sentry/react)	Optional client-side error capture
Backend (optional)	Firebase Cloud Functions (Node 20)	Callable/HTTP triggers, admin SDK (e.g. email, imports)—deploy separate from hosting
Infrastructure	Firebase Hosting	Serves dist/; SPA rewrite to index.html
Data rules	firestore.rules, storage.rules	Authorization for Firestore/Storage (not optional in production)
CI	GitHub Actions	Build/test/deploy pipelines (see .github/workflows/)
Tests	Vitest (unit), Playwright (e2e)	Quality gates

3. End-to-end workflow (how the site “works”)

3.1 First visit → marketing

1. User opens the hosted URL (e.g. Firebase Hosting).
2. **Firebase Hosting** returns `index.html` and static assets (`/assets/*`).
3. React boots from `src/main.jsx`, mounts `App.jsx`, and **React Router** renders routes like `/`, `/services`, `/map`, `/guarantee`, etc.
4. **No login required** for most marketing and map discovery flows.

3.2 Sign-up / sign-in

1. `AuthContext` wraps the app and subscribes to **Firebase Auth** (`onAuthStateChanged`).
2. Users can sign in with **email/password** or **Google** (per implementation in `AuthContext.jsx`).
3. After auth, profile and role data are loaded/created via **Firestore** (`userProfiles`, `userRoles`, etc.) and helpers in `profileService` / `firestoreStore` .
4. **Admin emails** can be elevated via env `VITE_ADMIN_EMAILS` (comma-separated) in addition to Firestore role docs.

3.3 Role-based areas

- `/customer`, `/seller`, `/admin` : wrapped in `RoleRoute` — UI checks **role** from context; mismatches redirect.
- `/crm` : `StaffRoute` for `admin`, `sub_admin`, `consultant` — staff CRM (leads, tasks, notifications).
- **Firestore rules** must align with these roles (e.g. `crmLeads` readable only by assigned staff or admins)—the **server-side** enforcement is rules, not the React wrapper alone.

3.4 Map & listings

1. `/map` loads `MapView` (Leaflet).
2. Listings are read from **Firestore** (and related collections) per app logic; **public** fields are safe to expose; **private** fields (e.g. broker phones) rely on **rules** and client helpers like `sanitizePublicListing` / `accessControl.js` .
3. Deep links and filters are handled via URL/query params as implemented in the map component.

3.5 Staff CRM (operational)

1. Staff opens `/crm` after sign-in.
2. **Leads** (`crmLeads`) and **tasks** (`crmTasks`) are read/written through `firestoreStore.js` .
3. **Sub-admins** and **consultants** are scoped to rows where `assigneeEmail` matches their account (rules + queries).
4. **Admins** see all leads and can reassign via staff picker; **bulk import** can paste TSV from Excel; `extraFields` preserves unknown columns.
5. **Notifications** collection drives in-app alerts for staff.

3.6 Deploy pipeline (typical)

1. `npm run build` → outputs static files to `dist/` .
2. `firebase deploy --only hosting` (often with `firestore:rules`) publishes the SPA and updates rules.
3. **Cloud Functions** are a **separate** deploy (`firebase deploy --only functions`); failures there do not roll back Hosting but leave features (e.g. callables) unavailable.

4. Repository layout (conceptual)

```
src/
  App.jsx           # Routes, RoleRoute, StaffRoute
  main.jsx          # Entry, legacy hash migration
  context/AuthContext.jsx
  lib/
    firebase.js     # Firebase app, auth, db, storage, functions, App Check
    firestoreStore.js # Firestore CRUD helpers
    accessControl.js # Role helpers aligned with rules intent
    crmSheetMapping.js # Excel → CRM payload + extraFields
  pages/            # Home, MapView, dashboards, CRM, etc.
  components/       # Map, layout, sections
  firestore.rules    # Security: who can read/write which collections
  storage.rules      # Storage access
  functions/         # Cloud Functions (Node 20, firebase-functions)
  firebase.json      # Hosting, Firestore, Storage, Functions config
```

5. Security model (short)

- **Authentication:** Firebase Auth (identity).
- **Authorization:** Primarily **Firestore Security Rules** and **Storage Rules** — must match product intent (staff vs customer vs seller).
- **App Check:** Optional reCAPTCHA v3 provider in `firebase.js` to reduce abuse of Firebase APIs.
- **Client-side role checks** (React) improve UX but **do not replace** rules.

6. Observability & quality

- **Sentry** can report client errors (configuration-dependent).
- `clientLog` / **reporting helpers** may warn on non-fatal issues (e.g. App Check init).
- **Vitest** / **Playwright** support regression testing; **ESLint** is configured but the full-repo lint may surface legacy debt—run scoped lint on changed files when iterating quickly.

7. Skills for building and maintaining this stack (2026)

Core (must-have)

- **Modern JavaScript + React** — hooks, router, async data loading, error boundaries.
- **Firebase Auth + Firestore** — security rules, indexes, query patterns, offline/cache behavior.
- **Git + CI** — branching, review, automated build/deploy.

Strongly recommended

- **Threat modeling for SPAs** — never trust the client; rules are the contract.
- **Basic performance** — bundle size, map performance, lazy loading where appropriate.
- **Testing** — smoke e2e for auth and critical paths; unit tests for pure logic (e.g. CRM mapping).

8. “Vibe coding” and common failure modes

Rapid AI-assisted development often ships UIs quickly but **under-invests** in the boring parts. For **this** architecture, the highest-risk gaps are:

Area	Risk	Mitigation mindset
Authorization	UI hides buttons but Firestore rules stay wide open	Rules review per collection; test with non-admin accounts
Authentication	Wrong session handling, missing email verification flows	Align AuthContext with product rules; test Google vs password
API / data layer	Treating Firestore like a REST API without handling permissions errors	User-visible errors, retries, and logging for permission-denied
Error logging	Silent catches; no Sentry DSN in prod	Centralize reporting; never swallow errors without telemetry
Cloud Functions	Deploy drift: hosting works, callable 404/timeout	Separate health checks; monitor function logs
Environment	Missing <code>VITE_*</code> vars in build	Document required env; validate at build time where possible
Indexes	New compound queries fail until indexes are created	Watch Firestore console links from error messages

Bottom line: The “skill” that separates production-grade work from vibe-coded demos is **verification**: read the rules, test as each role, and assume the client is hostile. AI can draft components; **humans** (or disciplined checklists) must validate security and failure paths.

9. Reference URLs & artifacts

- **Hosting:** Configured in `firebase.json` (SPA rewrites, cache headers).
- **Header exports for CRM sheets:** `docs/crm-all-header-rows.txt` and related `.tsv` files.
- **Deployment notes:** `docs/DEPLOYMENT.md`, `docs/PRODUCTION_GO_LIVE_CHECKLIST.md`.

End of document.